

PROGRAMMING WITH C++

UNIT 7

STANDARD LIBRARY CONTAINERS AND ITERATORS

Outlines

- Introduction to Big O
- Introduction to Hash Tables
- Introduction to Containers
- Working with Iterators
- Sequence Container (vector, list, deque, Associative)
- Container Adaptors
- Lambda Expression

Introduction to Big O

Big O notation is a mathematical concept used in computer science to describe the efficiency of an algorithm. It provides a way to classify algorithms based on their runtime or space requirements as the input size grows.

Why is Big O Important?

- Helps compare different algorithms.
- Allows developers to optimize code performance.
- Predicts how an algorithm behaves with large inputs.

Common Big O Notations

- Constant Time – $O(1)$
 - The runtime does not depend on the input size.
- Linear Time – $O(n)$
 - The runtime increases proportionally with the input size.
- Logarithmic Time – $O(\log n)$
 - The runtime grows logarithmically, common in search operations.
- Quadratic Time – $O(n^2)$
 - The runtime grows proportionally to the square of the input size.
- Exponential Time – $O(2^n)$
 - The runtime doubles with each additional input.

Constant Time – $O(1)$

- The runtime does not depend on the input size.

```
void constantTime(int arr[], int size) {  
    cout << arr[0] << endl; // Always takes the same amount of time.  
}
```

- Even if the array size increases, this function always runs in the same amount of time.

Linear Time – $O(n)$

- The runtime increases proportionally with the input size.

```
void linearTime(int arr[], int size) {  
    for (int i = 0; i < size; i++) {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
}
```

- If size doubles, the time taken doubles.

Logarithmic Time – $O(\log n)$

- The runtime grows logarithmically, common in search operations.

```
int binarySearch(int arr[], int left, int right, int target) {  
    while (left <= right) {  
        int mid = left + (right - left) / 2;  
        if (arr[mid] == target)  
            return mid;  
        else if (arr[mid] < target)  
            left = mid + 1;  
        else  
            right = mid - 1;  
    }  
    return -1;  
}
```

- Binary search reduces the search space in half each step, leading to **$O(\log n)$** complexity.

Quadratic Time – $O(n^2)$

- The runtime grows proportionally to the square of the input size.

```
void quadraticTime(int arr[], int size) {  
    for (int i = 0; i < size; i++) {  
        for (int j = 0; j < size; j++) {  
            cout << arr[i] << ", " << arr[j] << endl;  
        }  
    }  
}
```

- This is typical in nested loops, such as bubble sort.

Exponential Time – $O(2^n)$

- The runtime doubles with each additional input.

```
int fibonacci(int n) {  
    if (n <= 1) return n;  
    return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

- Recursive Fibonacci is an example of exponential complexity.

Hash Tables in C++

- A **hash table** (also known as a **hash map**) is a **data structure** that allows for efficient **key-value pair storage**.
- It is used for fast **data retrieval**, insertion, and deletion operations, typically with an **average time complexity of $O(1)$** .
- Hash tables are particularly useful when you need to associate a **unique key** with a value and retrieve data based on that key quickly.

Key Concepts of Hash Tables

- **Hash Function:**

- A **hash function** is a function that takes an input (or "key") and returns an index (or "hash code") in an array where the corresponding value is stored.
- The function aims to distribute the keys uniformly across the hash table, minimizing collisions (where two keys hash to the same index).

- **Buckets:**

- A **bucket** is a location in the hash table that stores values. Each bucket can hold multiple entries, typically managed through a **linked list** or another container.
- When a hash function generates the same hash for different keys, those keys will be placed in the same bucket, resulting in a **collision**.

- **Collisions:**

- **Collisions** happen when two different keys hash to the same bucket. There are two main ways to handle collisions:
 - **Chaining:** Each bucket holds a collection (such as a linked list) of elements that share the same hash code.
 - **Open Addressing:** All elements are stored directly in the array, and a new location is found if a collision occurs (usually through methods like linear probing, quadratic probing, or double hashing).

- **Load Factor:**

- The **load factor** of a hash table is defined as the number of elements divided by the number of buckets. It determines how "full" the hash table is.
- A high load factor may lead to performance degradation (more collisions), and the hash table may need **reallocation** (resize) to maintain optimal performance.

How Hash Tables Work

- **Insertion:**

- A key-value pair is inserted by computing the hash of the key and placing it in the corresponding bucket.
- If a collision occurs, the item is inserted into the appropriate location in the bucket.

- **Search:**

- To search for a value, the hash table computes the hash of the key and looks for the value in the corresponding bucket.
- If there are multiple items in the same bucket (due to a collision), a secondary search is performed within the bucket.

- **Deletion:**

- To delete a key-value pair, the hash table computes the hash of the key and removes the corresponding item from the bucket.
- If there is more than one item in the bucket, the appropriate entry is found and removed.

Advantages of Hash Tables

- **Efficient Lookup, Insertion, and Deletion:**

Hash tables provide **$O(1)$** time complexity for lookup, insertion, and deletion on average, making them very efficient for large datasets.

- **Fast Access:**

Direct access to an element using its key is possible due to the **hash function** that computes the location of the element.

- **Flexible Key-Value Mapping:**

Hash tables allow you to map a variety of **keys** (such as integers, strings, or custom types) to **values** (like integers, objects, etc.).

Hash Table Operations in C++

- In C++, the **std::unordered_map** and **std::unordered_set** are the standard hash table containers.
- These containers allow for efficient **key-value pair storage** and retrieval.
 - **std::unordered_map<Key, Value>**: Stores key-value pairs, where each key maps to a value.
 - **std::unordered_set<Key>**: Stores keys only, ensuring all keys are unique.

Example

```
#include <iostream>
#include <unordered_map>
#include <string>
using namespace std;

int main() {
    // Create a hash table (unordered_map)
    unordered_map<string, int> studentGrades;

    // Insert key-value pairs (key = student name, value = grade)
    studentGrades["Alice"] = 85;
    studentGrades["Bob"] = 92;
    studentGrades["Charlie"] = 78;

    // Search for a key and print the corresponding value
    string student = "Bob";
    if (studentGrades.find(student) != studentGrades.end()) {
        cout << student << "'s grade: " << studentGrades[student] <<
endl;
    } else {
        cout << student << " not found." << endl;
    }

    // Iterate through the hash table
    for (const auto& pair : studentGrades) {
        cout << pair.first << ": " << pair.second << endl;
    }

    return 0;
}
```

Output:

```
Bob's grade: 92
Charlie: 78
Bob: 92
Alice: 85
```

Introduction to Containers in C++

- Containers in C++ are data structures provided by the **Standard Template Library (STL)** that store collections of objects. They simplify memory management and improve efficiency in handling data.

C++ provides three main types of containers:

- **Sequence Containers** – Store elements in a linear order.
- **Associative Containers** – Store elements in a sorted or hashed manner for fast lookup.
- **Unordered Containers** – Use hashing for fast operations.

Sequence Containers

- Sequence containers in C++ **store elements in a linear order** and allow efficient traversal, insertion, and deletion.
- They are part of the **Standard Template Library (STL)** and are useful for managing collections of elements.

Container	Description
<code>std::vector</code>	Dynamic array, fast random access, efficient resizing.
<code>std::deque</code>	Double-ended queue, fast insertion/deletion at both ends.
<code>std::list</code>	Doubly linked list, efficient insert/delete anywhere.
<code>std::forward_list</code>	Singly linked list, lower memory usage.
<code>std::array</code>	Fixed-size array wrapper with STL compatibility.

`std::vector` in C++ (Dynamic Array)

- `std::vector` is a dynamic array that can automatically resize itself when elements are added or removed. It is part of the Standard Template Library (STL) and is one of the most commonly used sequence containers in C++.
- **Key Points of Vector:**
 - **Dynamic Sizing:** Grows automatically when needed.
 - **Fast Random Access:** $O(1)$ time complexity for element access.
 - **Efficient Insert/Remove at End:** $O(1)$ for push/pop at the back.
 - **Slow Insert/Remove at Front/Middle:** $O(n)$ due to shifting elements.

Declaring and Initializing std::vector

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> vec1; // Empty vector
    std::vector<int> vec2(5); // Vector of size 5 (default values)
    std::vector<int> vec3(5, 100); // Vector of size 5 with all values = 100
    std::vector<int> vec4 = {1, 2, 3, 4, 5}; // Initialization with values

    return 0;
}
```

Adding and Removing Elements

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> vec;

    vec.push_back(10); // Add 10 at the end
    vec.push_back(20);
    vec.push_back(30);

    vec.pop_back(); // Removes the last element (30)

    std::cout << "Vector elements: ";
    for (int num : vec) {
        std::cout << num << " ";
    }

    return 0;
}
```

Output:

```
Vector elements: 10 20
```

Accessing Elements

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> vec = {1, 2, 3, 4, 5};

    std::cout << "First Element: " << vec.front() << std::endl;
    std::cout << "Last Element: " << vec.back() << std::endl;
    std::cout << "Element at Index 2: " << vec[2] << std::endl;

    return 0;
}
```

Output:

```
First Element: 1
Last Element: 5
Element at Index 2: 3
```

Vector Capacity & Size Methods

Method	Description
<code>size()</code>	Returns the number of elements in the vector.
<code>capacity()</code>	Returns the total allocated capacity.
<code>max_size()</code>	Returns the maximum number of elements the vector can hold.
<code>resize(n)</code>	Resizes the vector to <code>n</code> elements (default initializes new elements).
<code>shrink_to_fit()</code>	Reduces the capacity to match the size.
<code>empty()</code>	Returns <code>true</code> if the vector is empty, otherwise <code>false</code> .

Vector Element Access Methods

Method	Description
<code>at(index)</code>	Returns the element at <code>index</code> (safe, with bounds checking).
<code>operator[]</code>	Returns the element at <code>index</code> (faster, but no bounds checking).
<code>front()</code>	Returns the first element of the vector.
<code>back()</code>	Returns the last element of the vector.
<code>data()</code>	Returns a pointer to the first element (raw array).

Modifiers (Insertion, Deletion, and Modification)

Method	Description
<code>push_back(value)</code>	Adds an element at the end.
<code>pop_back()</code>	Removes the last element.
<code>insert(position, value)</code>	Inserts an element at the specified position.
<code>insert(position, count, value)</code>	Inserts <code>count</code> copies of <code>value</code> at the position.
<code>insert(position, first, last)</code>	Inserts elements from another container/range.
<code>erase(position)</code>	Removes an element at the specified position.
<code>erase(first, last)</code>	Removes elements in the given range.
<code>clear()</code>	Removes all elements but retains allocated memory.
<code>assign(count, value)</code>	Assigns <code>count</code> copies of <code>value</code> to the vector.
<code>assign(first, last)</code>	Assigns values from another container/range.
<code>emplace(position, value)</code>	Inserts an element at the position (constructs in-place).
<code>emplace_back(value)</code>	Adds an element at the end (constructs in-place).
<code>swap(other)</code>	Swaps contents with another vector.

Iterators (Traversal & Access)

Method	Description
<code>begin()</code>	Returns an iterator to the first element.
<code>end()</code>	Returns an iterator to the element after the last one.
<code>rbegin()</code>	Returns a reverse iterator to the last element.
<code>rend()</code>	Returns a reverse iterator to before the first element.
<code>cbegin()</code>	Returns a constant iterator to the first element.
<code>cend()</code>	Returns a constant iterator to the element after the last one.
<code>crbegin()</code>	Returns a constant reverse iterator to the last element.
<code>crend()</code>	Returns a constant reverse iterator to before the first element.

Relational Operators (Comparisons)

Method	Description
<code>operator==</code>	Returns <code>true</code> if two vectors are equal.
<code>operator!=</code>	Returns <code>true</code> if two vectors are not equal.
<code>operator<</code>	Returns <code>true</code> if first vector is lexicographically smaller.
<code>operator></code>	Returns <code>true</code> if first vector is lexicographically greater.
<code>operator<=</code>	Returns <code>true</code> if first vector is smaller or equal.
<code>operator>=</code>	Returns <code>true</code> if first vector is greater or equal.

Example

```
#include <iostream>
#include <vector>
#include <algorithm> // for sort, find
using namespace std;

int main() {
    // Create an empty vector of integers
    vector<int> v;

    // push_back: Add elements
    v.push_back(10);
    v.push_back(20);
    v.push_back(30);

    // emplace_back: More efficient insertion
    v.emplace_back(40);

    // insert: Insert at a specific position
    v.insert(v.begin() + 2, 25); // insert 25 at index 2

    // erase: Remove element at index
    v.erase(v.begin() + 1); // remove element at index 1 (value 20)

    // pop_back: Remove last element
    v.pop_back(); // removes 40

    // size: Number of elements
    cout << "Size: " << v.size() << endl;

    // capacity: Current allocated memory
    cout << "Capacity: " << v.capacity() << endl;

    // front and back
```

```
    cout << "Front: " << v.front() << ", Back: " << v.back() << endl;

    // at: Safe element access
    cout << "Element at index 1: " << v.at(1) << endl;

    // resize: Change number of elements
    v.resize(6, 100); // fill new slots with 100

    // sort
    sort(v.begin(), v.end());

    // find
    auto it = find(v.begin(), v.end(), 100);
    if (it != v.end()) {
        cout << "Found 100 at index: " << distance(v.begin(), it) << endl;
    }

    // clear: Remove all elements
    // v.clear();

    // empty: Check if vector is empty
    cout << "Is vector empty? " << (v.empty() ? "Yes" : "No") << endl;

    // Display vector
    cout << "Vector elements: ";
    for (int x : v) {
        cout << x << " ";
    }
    cout << endl;

    return 0;
}
```

Output:

```
Size: 3
Capacity: 8
Front: 10, Back: 30
Element at index 1: 25
Found 100 at index: 3
Is vector empty? No
Vector elements: 10 25 30 100 100 100
```

`std::deque` (Double-Ended Queue) in C++

- `std::deque` (Double-Ended Queue) is a sequence container in C++ that allows fast insertion and deletion at both ends (front and back).
- It is similar to `std::vector`, but provides efficient operations at both ends, whereas `std::vector` is optimized for operations at the back only.
- **Key points of `std::deque`**
 - Fast Insertions/Deletions at Front & Back: $O(1)$ for `push_front()` and `push_back()`.
 - Efficient Random Access: $O(1)$ time complexity for indexing (`operator[]`).
 - Dynamic Resizing: No need to manually manage memory.
 - Slower Middle Insertions/Deletions: $O(n)$ due to shifting elements.

Declaring and Initializing std::deque

```
#include <iostream>
#include <deque>

int main() {
    std::deque<int> dq1; // Empty deque
    std::deque<int> dq2(5); // Deque of size 5 (default values)
    std::deque<int> dq3(5, 100); // Deque of size 5 with all values = 100
    std::deque<int> dq4 = {1, 2, 3, 4, 5}; // Initialization with values

    return 0;
}
```

Adding and Removing Elements

```
#include <iostream>
#include <deque>

int main() {
    std::deque<int> dq;

    dq.push_back(10); // Add at the end
    dq.push_front(20); // Add at the front
    dq.push_back(30);
    dq.push_front(40);

    dq.pop_back(); // Remove last element (30)
    dq.pop_front(); // Remove first element (40)

    std::cout << "Deque elements: ";
    for (int num : dq) {
        std::cout << num << " ";
    }

    return 0;
}
```

Output:

```
Deque elements: 20 10
```

Accessing Elements

```
#include <iostream>
#include <deque>

int main() {
    std::deque<int> dq = {1, 2, 3, 4, 5};

    std::cout << "First Element: " << dq.front() << std::endl;
    std::cout << "Last Element: " << dq.back() << std::endl;
    std::cout << "Element at Index 2: " << dq[2] << std::endl;

    return 0;
}
```

Output:

```
First Element: 1
Last Element: 5
Element at Index 2: 3
```

Methods of `std::deque`

Category	Method	Description
Capacity & Size	<code>size()</code>	Returns the number of elements in the deque.
	<code>max_size()</code>	Returns the maximum number of elements the deque can hold.
	<code>resize(n)</code>	Resizes the deque to <code>n</code> elements (default initializes new elements).
	<code>shrink_to_fit()</code>	Reduces memory usage.
	<code>empty()</code>	Returns <code>true</code> if the deque is empty, otherwise <code>false</code> .
Element Access	<code>at(index)</code>	Returns the element at <code>index</code> (safe, with bounds checking).
	<code>operator[]</code>	Returns the element at <code>index</code> (faster, but no bounds checking).
	<code>front()</code>	Returns the first element of the deque.
	<code>back()</code>	Returns the last element of the deque.

Methods of `std::deque`

Category	Method	Description
Modifiers (Insertion & Deletion)	<code>push_back(value)</code>	Adds an element at the end.
	<code>push_front(value)</code>	Adds an element at the front.
	<code>pop_back()</code>	Removes the last element.
	<code>pop_front()</code>	Removes the first element.
	<code>insert(position, value)</code>	Inserts an element at the specified position.
	<code>erase(position)</code>	Removes an element at the specified position.
	<code>clear()</code>	Removes all elements but retains allocated memory.
	<code>swap(other)</code>	Swaps contents with another deque.
Iterators (Traversal & Access)	<code>begin()</code>	Returns an iterator to the first element.
	<code>end()</code>	Returns an iterator to the element after the last one.
	<code>rbegin()</code>	Returns a reverse iterator to the last element.
	<code>rend()</code>	Returns a reverse iterator to before the first element.

Example

```
#include <iostream>
#include <deque>
#include <algorithm> // for sort, find
using namespace std;

int main() {
    // Create a deque of integers
    deque<int> dq;

    // push_back and push_front
    dq.push_back(10);
    dq.push_back(20);
    dq.push_front(5);
    dq.push_front(1);

    // emplace_back and emplace_front
    dq.emplace_back(25);
    dq.emplace_front(0);

    // at, front, back
    cout << "Element at index 2: " << dq.at(2) << endl;
    cout << "Front: " << dq.front() << ", Back: " << dq.back() << endl;

    // pop_back and pop_front
    dq.pop_back();
    dq.pop_front();

    // insert: insert 99 at position 2
    dq.insert(dq.begin() + 2, 99);

    // erase: remove element at position 3
    dq.erase(dq.begin() + 3);

    // size and empty
    cout << "Size: " << dq.size() << endl;
    cout << "Is deque empty? " << (dq.empty() ? "Yes" : "No") << endl;

    // sort (only works with random access iterators)
    sort(dq.begin(), dq.end());

    // find
    auto it = find(dq.begin(), dq.end(), 99);
    if (it != dq.end()) {
        cout << "Found 99 at index: " << distance(dq.begin(), it) << endl;
    }

    // Display deque
    cout << "Deque elements: ";
    for (int x : dq) {
        cout << x << " ";
    }
    cout << endl;

    return 0;
}
```

Output:

```
Element at index 2: 5
Front: 0, Back: 25
Size: 4
Is deque empty? No
Found 99 at index: 3
Deque elements: 1 5 20 99
```

`std::list` in C++

- `std::list` is a doubly linked list container in C++ that allows fast insertions and deletions at any position.
- Unlike `std::vector` and `std::deque`, `std::list` does not support direct access via indexing (`[]`).
- **Key points of `std::list`**
 - Efficient insertions and deletions at any position ($O(1)$).
 - Dynamic memory allocation (no pre-allocation needed).
 - Supports forward and backward traversal (doubly linked list).
 - No direct random access ($O(n)$ time to access an element at index i).

Declaring and Initializing std::list

```
#include <iostream>
#include <list>

int main() {
    std::list<int> lst1; // Empty list
    std::list<int> lst2(5); // List of size 5 (default values)
    std::list<int> lst3(5, 100); // List of size 5 with all values
    = 100
    std::list<int> lst4 = {1, 2, 3, 4, 5}; // Initialization with
    values

    return 0;
}
```

Adding and Removing Elements

```
#include <iostream>
#include <list>

int main() {
    std::list<int> lst;

    lst.push_back(10); // Add at the end
    lst.push_front(20); // Add at the front
    lst.push_back(30);
    lst.push_front(40);

    lst.pop_back(); // Remove last element (30)
    lst.pop_front(); // Remove first element (40)

    std::cout << "List elements: ";
    for (int num : lst) {
        std::cout << num << " ";
    }

    return 0;
}
```

Output:

```
List elements: 20 10
```

Accessing Elements

- `std::list` does not support direct indexing (`[]`) or `.at()`. Use iterators instead:

```
#include <iostream>
#include <list>

int main() {
    std::list<int> lst = {1, 2, 3, 4, 5};

    std::cout << "First Element: " << lst.front() << std::endl;
    std::cout << "Last Element: " << lst.back() << std::endl;

    return 0;
}
```

Output:

```
First Element: 1
Last Element: 5
```

Methods of std::list

Category	Method	Description
Capacity & Size	<code>size()</code>	Returns the number of elements in the list.
	<code>max_size()</code>	Returns the maximum number of elements the list can hold.
	<code>resize(n)</code>	Resizes the list to <code>n</code> elements (default initializes new elements).
	<code>empty()</code>	Returns <code>true</code> if the list is empty, otherwise <code>false</code> .
Element Access	<code>front()</code>	Returns the first element of the list.
	<code>back()</code>	Returns the last element of the list.
Modifiers (Insertion & Deletion)	<code>push_back(value)</code>	Adds an element at the end.
	<code>push_front(value)</code>	Adds an element at the front.
	<code>pop_back()</code>	Removes the last element.
	<code>pop_front()</code>	Removes the first element.
	<code>insert(position, value)</code>	Inserts an element at the specified position.
	<code>erase(position)</code>	Removes an element at the specified position.
	<code>clear()</code>	Removes all elements but retains allocated memory.
	<code>swap(other)</code>	Swaps contents with another list.

Methods of std::list

Category	Method	Description
Iterators (Traversal & Access)	<code>begin()</code>	Returns an iterator to the first element.
	<code>end()</code>	Returns an iterator to the element after the last one.
	<code>rbegin()</code>	Returns a reverse iterator to the last element.
	<code>rend()</code>	Returns a reverse iterator to before the first element.
Special Operations	<code>remove(value)</code>	Removes all elements with the given value.
	<code>remove_if(predicate)</code>	Removes elements that satisfy a condition.
	<code>unique()</code>	Removes consecutive duplicate elements.
	<code>reverse()</code>	Reverses the order of elements.
	<code>sort()</code>	Sorts the list in ascending order.
	<code>merge(other_list)</code>	Merges two sorted lists.
	<code>splice(position, other_list)</code>	Transfers elements from another list at a specified position.

Example

```
#include <iostream>
#include <list>
#include <algorithm> // for find, sort
using namespace std;

int main() {
    // Create a list of integers
    list<int> lst;

    // push_back and push_front
    lst.push_back(10);
    lst.push_back(20);
    lst.push_front(5);
    lst.push_front(1);

    // emplace_back and emplace_front
    lst.emplace_back(30);
    lst.emplace_front(0);

    // front and back
    cout << "Front: " << lst.front() << ", Back: " << lst.back() << endl;

    // pop_back and pop_front
    lst.pop_back();
    lst.pop_front();

    // insert (only supports insertion using iterator)
    auto it = lst.begin();
    advance(it, 2); // move iterator to 3rd position
    lst.insert(it, 99);

    // erase (using iterator)
    advance(it, 3); // move to 4th element
    lst.erase(it);

    // size and empty
    cout << "Size: " << lst.size() << endl;
    cout << "Is list empty? " << (lst.empty() ? "Yes" : "No") << endl;

    // find
    auto findIt = find(lst.begin(), lst.end(), 99);
    if (findIt != lst.end()) {
        cout << "Found 99 in list." << endl;
    }

    // sort (only works on list itself, not from <algorithm>)
    lst.sort();

    // reverse
    lst.reverse();

    // remove specific value (e.g., remove 10)
    lst.remove(10);

    // Display list
    cout << "List elements: ";
    for (int x : lst) {
        cout << x << " ";
    }
    cout << endl;

    return 0;
}
```

Output:

```
Front: 0, Back: 30
Size: 4
Is list empty? No
Found 99 in list.
List elements: 99 20 5 1
```

std::forward_list in C++

- std::forward_list is a singly linked list container in C++ that allows fast insertions and deletions but only supports forward traversal.
- Unlike std::list, it does not support backward traversal since it is singly linked.
- **Key Points of std::forward_list**
 - Efficient insertions and deletions at any position ($O(1)$).
 - Uses less memory compared to std::list (no extra backward pointers).
 - Faster than std::list in certain cases due to lower memory overhead.
 - No direct access via indexing ([] or .at()).
 - No size() function (requires manual counting).
 - Only supports forward iteration (no rbegin() or rend()).

Declaring and Initializing std::forward_list

```
#include <iostream>
#include <forward_list>

int main() {
    std::forward_list<int> flist1; // Empty list
    std::forward_list<int> flist2 = {1, 2, 3, 4, 5}; // Initialization with values
    std::forward_list<int> flist3(5, 100); // List of size 5 with all values = 100

    return 0;
}
```

Adding and Removing Elements

```
#include <iostream>
#include <forward_list>

int main() {
    std::forward_list<int> flist = {10, 20, 30};

    flist.push_front(5); // Add at the front
    flist.pop_front();   // Remove the first element

    std::cout << "Forward List elements: ";
    for (int num : flist) {
        std::cout << num << " ";
    }

    return 0;
}
```

Output:

```
Forward List elements: 10 20 30
```

Methods of `std::forward_list`

Category	Method	Description
Element Access	<code>front()</code>	Returns the first element.
Modifiers (Insertion & Deletion)	<code>push_front(value)</code>	Adds an element at the front.
	<code>pop_front()</code>	Removes the first element.
	<code>insert_after(position, value)</code>	Inserts an element after a given position.
	<code>erase_after(position)</code>	Removes the element after a given position.
	<code>remove(value)</code>	Removes all elements with the given value.
	<code>remove_if(predicate)</code>	Removes elements that satisfy a condition.
	<code>clear()</code>	Removes all elements.
Iterators (Traversal & Access)	<code>begin()</code>	Returns an iterator to the first element.
	<code>end()</code>	Returns an iterator to the element after the last one.
	<code>before_begin()</code>	Returns an iterator to the position before the first element.

Methods of std::forward_list

Category	Method	Description
Special Operations	<code>reverse()</code>	Reverses the order of elements.
	<code>sort()</code>	Sorts the list in ascending order.
	<code>unique()</code>	Removes consecutive duplicate elements.
	<code>merge(other_list)</code>	Merges two sorted lists.
	<code>splice_after(position, other_list)</code>	Transfers elements from another list after a given position.

Example

```
#include <iostream>
#include <forward_list>
#include <algorithm> // for find
using namespace std;

int main() {
    // Create a forward_list of integers
    forward_list<int> flist = {10, 20, 30};

    // push_front: Add elements to the front
    flist.push_front(5);
    flist.push_front(1);

    // emplace_front: Efficient in-place construction
    flist.emplace_front(0);

    // front: Access first element
    cout << "Front: " << flist.front() << endl;

    // insert_after: Insert after a given position
    auto it = flist.begin();
    advance(it, 2); // Move to the 3rd element
    flist.insert_after(it, 15); // Insert 15 after the 3rd element

    // emplace_after
    flist.emplace_after(it, 12); // Insert 12 after the 3rd element

    // erase_after: Remove element after a given position
    it = flist.begin();
    advance(it, 3);
    flist.erase_after(it); // Erase element after the 4th element

    // remove: Remove elements by value
    flist.remove(30); // Remove all occurrences of 30

    // sort
    flist.sort();

    // unique: Remove consecutive duplicates
    flist.unique();

    // reverse
    flist.reverse();

    // Display the forward_list
    cout << "Forward list elements: ";
    for (int x : flist) {
        cout << x << " ";
    }
    cout << endl;

    // Check if list is empty
    cout << "Is list empty? " << (flist.empty() ? "Yes" : "No") << endl;

    return 0;
}
```

Output:

```
Front: 0
Forward list elements: 20 12 10 5 1 0
Is list empty? No
```

Associative Containers in C++

- Associative containers in C++ are data structures that store elements in a way that allows fast searching, insertion, and deletion.
- They automatically maintain a sorted order (except for unordered_ versions).
- Types of Associative Containers:
 - Ordered Containers (Sorted by keys using a balanced binary tree - Red-Black Tree):
 - `std::set` (Unique sorted elements)
 - `std::map` (Key-value pairs, unique keys)
 - `std::multiset` (Allows duplicate elements)
 - `std::multimap` (Allows duplicate keys)
 - Unordered Containers (Fast access using Hash Tables):
 - `std::unordered_set`
 - `std::unordered_map`
 - `std::unordered_multiset`
 - `std::unordered_multimap`

std::set in C++

- A std::set is an ordered collection of unique elements.
- It is implemented as a balanced binary search tree (Red-Black Tree), ensuring efficient operations with $O(\log n)$ time complexity for insertions, deletions, and searches.
- Key points of std::set
 - Stores unique elements (No duplicates allowed).
 - Automatically sorts elements in ascending order (default).
 - Fast insert, erase, and find operations ($O(\log n)$).
 - No direct access via indexing ([] or .at()).

Declaring and Initializing a std::set

```
#include <iostream>
#include <set>

int main() {
    std::set<int> s1; // Empty set
    std::set<int> s2 = {10, 30, 20, 10, 40}; // Initialization with values
    std::set<int> s3(s2); // Copy constructor

    return 0;
}
```

Inserting and Erasing Elements

```
#include <iostream>
#include <set>

int main() {
    std::set<int> s;

    s.insert(10); // Insert 10
    s.insert(20);
    s.insert(30);
    s.insert(10); // Duplicate, won't be added

    s.erase(20); // Removes 20

    std::cout << "Set elements: ";
    for (int num : s) {
        std::cout << num << " ";
    }
}
```

Output:

```
Set elements: 10 30
```

Methods of std::set

Category	Method	Description
Insertion & Deletion	<code>insert(value)</code>	Inserts a value (no duplicates).
	<code>erase(value)</code>	Removes a specific value.
	<code>erase(iterator)</code>	Removes an element using an iterator.
	<code>clear()</code>	Deletes all elements.
Access & Iterators	<code>begin()</code>	Returns iterator to the smallest element.
	<code>end()</code>	Returns iterator past the last element.
	<code>rbegin()</code>	Reverse iterator to the largest element.
	<code>rend()</code>	Reverse iterator past the first element.
Search & Count	<code>find(value)</code>	Returns iterator to element if found, else <code>end()</code> .
	<code>count(value)</code>	Returns 1 if element exists, else 0.
	<code>lower_bound(value)</code>	Returns iterator to first element $\geq$ <code>value</code> .
	<code>upper_bound(value)</code>	Returns iterator to first element $>$ <code>value</code> .
Properties	<code>size()</code>	Returns number of elements.
	<code>empty()</code>	Returns <code>true</code> if set is empty.

Example

```
#include <iostream>
#include <set>
#include <algorithm> // for find (optional)
using namespace std;

int main() {
    // Create a set of integers
    set<int> s;

    // insert: Add elements
    s.insert(10);
    s.insert(5);
    s.insert(20);
    s.insert(15);
    s.insert(10); // Duplicate, will be ignored

    // emplace: More efficient insertion
    s.emplace(25);

    // size and empty
    cout << "Size: " << s.size() << endl;
    cout << "Is set empty? " << (s.empty() ? "Yes" : "No") << endl;

    // find
    auto it = s.find(15);
    if (it != s.end()) {
        cout << "Found: " << *it << endl;
    } else {
        cout << "15 not found." << endl;
    }
}
```

```
// count (returns 1 if element exists, 0 otherwise)
cout << "Is 10 in the set? " << (s.count(10) ? "Yes" : "No") << endl;

// erase by value
s.erase(20);

// erase by iterator
auto it2 = s.find(5);
if (it2 != s.end()) {
    s.erase(it2);
}

// lower_bound and upper_bound
cout << "Lower bound of 12: ";
auto lb = s.lower_bound(12);
if (lb != s.end()) cout << *lb << endl; else cout << "No such element\n";

cout << "Upper bound of 15: ";
auto ub = s.upper_bound(15);
if (ub != s.end()) cout << *ub << endl; else cout << "No such element\n";

// Display elements (auto-sorted in ascending order)
cout << "Set elements: ";
for (int x : s) {
    cout << x << " ";
}
cout << endl;

return 0;
}
```

Output:

```
Size: 5
Is set empty? No
Found: 15
Is 10 in the set? Yes
Lower bound of 12: 15
Upper bound of 15: 25
Set elements: 10 15 25
```

std::map

- A std::map is an ordered associative container that stores key-value pairs, where:
 - Keys are unique (No duplicates allowed).
 - Values are associated with keys (Each key maps to a value).
 - Elements are automatically sorted by keys.
 - Implemented as a self-balancing binary search tree (Red-Black Tree), providing $O(\log n)$ insert, erase, and search operations.

Declaring and Initializing a `std::map`

```
#include <iostream>
#include <map>

int main() {
    std::map<int, std::string> myMap; // Empty map

    std::map<int, std::string> fruits = {
        {1, "Apple"},
        {3, "Banana"},
        {2, "Cherry"}
    };

    return 0;
}
```

Inserting Elements

```
#include <iostream>
#include <map>

int main() {
    std::map<int, std::string> myMap;

    // Method 1: Using `insert()`
    myMap.insert({1, "Apple"});
    myMap.insert({3, "Banana"});

    // Method 2: Using `operator[]`
    myMap[2] = "Cherry";
    myMap[4] = "Dates";

    for (auto &pair : myMap) {
        std::cout << pair.first << " -> " << pair.second << std::endl;
    }
}
```

Output:

```
1 -> Apple
2 -> Cherry
3 -> Banana
4 -> Dates
```

Methods of std::map

Category	Method	Description
Insertion & Deletion	<code>insert({key, value})</code>	Inserts a key-value pair.
	<code>erase(key)</code>	Removes a key-value pair.
	<code>clear()</code>	Deletes all elements.
Access & Iterators	<code>begin()</code>	Returns iterator to the first element.
	<code>end()</code>	Returns iterator past the last element.
	<code>rbegin()</code>	Reverse iterator to the last element.
	<code>rend()</code>	Reverse iterator past the first element.
Search & Count	<code>find(key)</code>	Returns iterator to key if found, else <code>end()</code> .
	<code>count(key)</code>	Returns <code>1</code> if key exists, else <code>0</code> .
	<code>lower_bound(key)</code>	Returns iterator to first key $\geq$ <code>key</code> .
	<code>upper_bound(key)</code>	Returns iterator to first key $>$ <code>key</code> .
Properties	<code>size()</code>	Returns number of elements.
	<code>empty()</code>	Returns <code>true</code> if map is empty.

Example

```
#include <iostream>
#include <map>
#include <string>
using namespace std;

int main() {
    // Create a map with string keys and int values
    map<string, int> studentMarks;

    // insert elements using insert()
    studentMarks.insert({"Alice", 85});
    studentMarks.insert(make_pair("Bob", 90));

    // insert or update using [] operator
    studentMarks["Charlie"] = 78;
    studentMarks["David"] = 92;

    // emplace: more efficient insertion
    studentMarks.emplace("Eve", 88);

    // Access elements
    cout << "Alice's Marks: " << studentMarks["Alice"] << endl;

    // find
    auto it = studentMarks.find("Bob");
    if (it != studentMarks.end()) {
```

```
        cout << "Bob's Marks: " << it->second << endl;
    }

    // count (returns 1 if key exists, 0 otherwise)
    cout << "Is 'David' in map? " << (studentMarks.count("David") ? "Yes" : "No") << endl;

    // erase by key
    studentMarks.erase("Charlie");

    // size and empty
    cout << "Size of map: " << studentMarks.size() << endl;
    cout << "Is map empty? " << (studentMarks.empty() ? "Yes" : "No") << endl;

    // iterate through map (keys are auto-sorted)
    cout << "All student marks:" << endl;
    for (const auto& pair : studentMarks) {
        cout << pair.first << ": " << pair.second << endl;
    }

    // clear
    // studentMarks.clear();

    return 0;
}
```

Output:

```
Alice's Marks: 85
Bob's Marks: 90
Is 'David' in map? Yes
Size of map: 4
Is map empty? No
All student marks:
Alice: 85
Bob: 90
David: 92
Eve: 88
```

std::multiset

- A std::multiset is an ordered associative container similar to std::set, but with duplicate elements allowed.
 - Stores elements in sorted order (ascending by default).
 - Allows duplicate elements.
 - Implemented as a self-balancing binary search tree (Red-Black Tree).
 - Operations (insert, erase, search) take $O(\log n)$ time.

Declaring and Initializing a `std::multiset`

```
#include <iostream>
#include <set>

int main() {
    std::multiset<int> ms1; // Empty multiset
    std::multiset<int> ms2 = {10, 30, 20, 10, 40}; //
    Initialization with duplicates

    return 0;
}
```

Inserting Elements

```
#include <iostream>
#include <set>

int main() {
    std::multiset<int> ms;

    ms.insert(10);
    ms.insert(20);
    ms.insert(30);
    ms.insert(10); // Duplicate allowed

    std::cout << "Multiset elements: ";
    for (int num : ms) {
        std::cout << num << " ";
    }
}
```

Output:

Multiset elements: 10 10 20 30

Methods of std::multiset

Category	Method	Description
Insertion & Deletion	<code>insert(value)</code>	Inserts a value (duplicates allowed).
	<code>erase(value)</code>	Removes all occurrences of a value.
	<code>erase(iterator)</code>	Removes an element using an iterator.
	<code>clear()</code>	Deletes all elements.
Access & Iterators	<code>begin()</code>	Returns iterator to the first element.
	<code>end()</code>	Returns iterator past the last element.
	<code>rbegin()</code>	Reverse iterator to the last element.
	<code>rend()</code>	Reverse iterator past the first element.
Search & Count	<code>find(value)</code>	Returns iterator to first occurrence of value.
	<code>count(value)</code>	Returns number of occurrences of value.
	<code>lower_bound(value)</code>	Returns iterator to first element $\geq$ <code>value</code> .
	<code>upper_bound(value)</code>	Returns iterator to first element $>$ <code>value</code> .
Properties	<code>size()</code>	Returns number of elements.
	<code>empty()</code>	Returns <code>true</code> if multiset is empty.

Example

```
#include <iostream>
#include <set>
using namespace std;

int main() {
    // Create a multiset of integers
    multiset<int> ms;

    // insert (allows duplicates)
    ms.insert(10);
    ms.insert(20);
    ms.insert(10); // duplicate
    ms.insert(30);
    ms.insert(20); // duplicate

    // emplace
    ms.emplace(25);

    // size and empty
    cout << "Size: " << ms.size() << endl;
    cout << "Is multiset empty? " << (ms.empty() ? "Yes" : "No") << endl;

    // count: how many times an element appears
    cout << "Count of 10: " << ms.count(10) << endl;
    cout << "Count of 20: " << ms.count(20) << endl;

    // find: gets an iterator to *one* instance of the value
    auto it = ms.find(20);
    if (it != ms.end()) {
        cout << "Found: " << *it << endl;
    }
}
```

```
// erase by value: removes *all* instances of that value
ms.erase(10); // removes all 10s

// erase by iterator: removes just one occurrence
it = ms.find(20);
if (it != ms.end()) {
    ms.erase(it); // removes one 20
}

// lower_bound and upper_bound
cout << "Lower bound of 20: ";
auto lb = ms.lower_bound(20);
if (lb != ms.end()) cout << *lb << endl; else cout << "No element\n";

cout << "Upper bound of 20: ";
auto ub = ms.upper_bound(20);
if (ub != ms.end()) cout << *ub << endl; else cout << "No element\n";

// Display elements (sorted, including duplicates)
cout << "Multiset elements: ";
for (int x : ms) {
    cout << x << " ";
}
cout << endl;

return 0;
}
```

Output:

```
Size: 6
Is multiset empty? No
Count of 10: 2
Count of 20: 2
Found: 20
Lower bound of 20: 20
Upper bound of 20: 25
Multiset elements: 20 25 30
```

std::multimap

- A std::multimap is an ordered associative container similar to std::map, but with duplicate keys allowed.
 - Stores key-value pairs in sorted order (ascending by default).
 - Allows multiple values for the same key.
 - Implemented as a self-balancing binary search tree (Red-Black Tree).
 - Operations (insert, erase, search) take $O(\log n)$ time.

Declaring and Initializing a `std::multimap`

```
#include <iostream>
#include <map>

int main() {
    std::multimap<int, std::string> mm; // Empty multimap

    std::multimap<int, std::string> fruits = {
        {1, "Apple"},
        {3, "Banana"},
        {2, "Cherry"},
        {1, "Apricot"} // Duplicate key allowed
    };

    return 0;
}
```

Inserting Elements

```
#include <iostream>
#include <map>

int main() {
    std::multimap<int, std::string> mm;

    mm.insert({1, "Apple"});
    mm.insert({2, "Banana"});
    mm.insert({1, "Apricot"}); // Duplicate key

    for (const auto &pair : mm) {
        std::cout << pair.first << " -> " << pair.second << std::endl;
    }
}
```

Output:

```
1 -> Apple
1 -> Apricot
2 -> Banana
```

Methods of `std::multimap`

Category	Method	Description
Insertion & Deletion	<code>insert({key, value})</code>	Inserts a key-value pair (duplicates allowed).
	<code>erase(key)</code>	Removes all occurrences of a key.
	<code>erase(iterator)</code>	Removes an element using an iterator.
	<code>clear()</code>	Deletes all elements.
Access & Iterators	<code>begin()</code>	Returns iterator to the first element.
	<code>end()</code>	Returns iterator past the last element.
	<code>rbegin()</code>	Reverse iterator to the last element.
	<code>rend()</code>	Reverse iterator past the first element.
Search & Count	<code>find(key)</code>	Returns iterator to first occurrence of key.
	<code>count(key)</code>	Returns number of occurrences of key.
	<code>lower_bound(key)</code>	Returns iterator to first key $\geq$ key .
	<code>upper_bound(key)</code>	Returns iterator to first key $>$ key .
	<code>equal_range(key)</code>	Returns pair of iterators (first and last occurrence of key).
Properties	<code>size()</code>	Returns number of elements.
	<code>empty()</code>	Returns <code>true</code> if multimap is empty.

Example

```
#include <iostream>
#include <map>
#include <string>
using namespace std;

int main() {
    // Create a multimap with string keys and int values
    multimap<string, int> marks;

    // insert pairs (allows duplicate keys)
    marks.insert({"Alice", 85});
    marks.insert({"Bob", 90});
    marks.insert({"Alice", 88}); // duplicate key
    marks.insert({"Charlie", 75});

    // emplace (construct in-place)
    marks.emplace("Bob", 93);

    // size and empty
    cout << "Size: " << marks.size() << endl;
    cout << "Is multimap empty? " << (marks.empty() ? "Yes" : "No") << endl;

    // find: returns iterator to the *first* element with given key
    auto it = marks.find("Bob");
    if (it != marks.end()) {
        cout << "Found Bob: " << it->second << endl;
    }
}
```

```
// count: how many entries have the key
cout << "Count of 'Alice': " << marks.count("Alice") << endl;

// equal_range: get range of elements with a given key
auto range = marks.equal_range("Bob");
cout << "All marks for Bob: ";
for (auto itr = range.first; itr != range.second; ++itr) {
    cout << itr->second << " ";
}
cout << endl;

// erase: by key (removes all entries with that key)
marks.erase("Charlie");

// Display all key-value pairs (keys are sorted, duplicates allowed)
cout << "All student marks:" << endl;
for (const auto& p : marks) {
    cout << p.first << " => " << p.second << endl;
}

return 0;
}
```

Output:

```
Size: 5
Is multimap empty? No
Found Bob: 90
Count of 'Alice': 2
All marks for Bob: 90 93
All student marks:
Alice => 85
Alice => 88
Bob => 90
Bob => 93
```

std::unordered_set

std::unordered_multiset

- C++ provides unordered associative containers that use hash tables for fast operations.
 - std::unordered_set → Stores unique elements (no duplicates allowed).
 - std::unordered_multiset → Stores duplicate elements (allows multiple occurrences of the same value).

Feature	std::unordered_set	std::unordered_multiset
Duplicate Elements Allowed?	✗ No	✓ Yes
Order Maintained?	✗ No	✗ No
Implementation	Hash Table	Hash Table
Insertion Complexity	$O(1)$ (amortized)	$O(1)$ (amortized)
Search Complexity	$O(1)$ (amortized)	$O(1)$ (amortized)
Best Use Case	Fast lookup of unique elements	Counting occurrences of elements

Example of std::unordered_set

```
#include <iostream>
#include <unordered_set>

int main() {
    std::unordered_set<int> us = {10, 20, 30, 10}; // Duplicates ignored

    for (int num : us) {
        std::cout << num << " "; // Order not guaranteed
    }
}
```

Output:

30 20 10

Example of `std::unordered_multiset`

```
#include <iostream>
#include <unordered_multiset>

int main() {
    std::unordered_multiset<int> ums = {10, 20, 30, 10, 20};

    for (int num : ums) {
        std::cout << num << " ";
    }
}
```

Output:

30 20 20 10 10

std::unordered_map

std::unordered_multimap

- C++ provides unordered associative containers that store key-value pairs in hash tables for fast operations.
 - std::unordered_map → Stores unique keys, each key maps to a single value.
 - std::unordered_multimap → Stores duplicate keys, each key can have multiple values.

Feature	std::unordered_map	std::unordered_multimap
Duplicate Keys Allowed?	✗ No	✓ Yes
Multiple Values per Key?	✗ No	✓ Yes
Ordering Guaranteed?	✗ No	✗ No
Implementation	Hash Table	Hash Table
Insertion Complexity	$O(1)$ (amortized)	$O(1)$ (amortized)
Search Complexity	$O(1)$ (amortized)	$O(1)$ (amortized)
Best Use Case	Fast key-value lookup	Storing multiple values per key

Example of std::unordered_map

```
#include <iostream>
#include <unordered_map>

int main() {
    std::unordered_map<int, std::string> um = {{1, "Apple"}, {2, "Banana"}, {3, "Cherry"}};

    for (const auto &pair : um) {
        std::cout << pair.first << " -> " << pair.second << std::endl;
    }
}
```

Output:

```
3 -> Cherry
2 -> Banana
1 -> Apple
```

Example of std::unordered_multimap

```
#include <iostream>
#include <unordered_map>

int main() {
    std::unordered_multimap<int, std::string> umm = {
        {1, "Apple"}, {2, "Banana"}, {1, "Apricot"}, {3, "Cherry"}
    };

    for (const auto &pair : umm) {
        std::cout << pair.first << " -> " << pair.second << std::endl;
    }
}
```

Output:

```
3 -> Cherry
2 -> Banana
1 -> Apricot
1 -> Apple
```

Working with Iterators in C++

- **Iterators** in C++ are used to traverse and manipulate elements in **containers** (such as vectors, lists, maps, etc.). They are an essential part of the **Standard Template Library (STL)**, providing a uniform interface for working with different container types.
- Iterators act like **pointers** that can be used to **access elements**, **move through the container**, and **perform operations** on the elements without directly manipulating the underlying data structure.

Type of iterator

Iterator Type	Description	Example Container
Input Iterator	Can read data in one direction (cannot write or modify data).	<code>std::istream_iterator</code>
Output Iterator	Can write data to a container (cannot read data).	<code>std::ostream_iterator</code>
Forward Iterator	Can read and write data in one direction.	<code>std::list</code> , <code>std::forward_list</code>
Bidirectional Iterator	Can move forward and backward.	<code>std::deque</code> , <code>std::list</code>
Random Access Iterator	Can move freely forward and backward, and supports indexing.	<code>std::vector</code> , <code>std::array</code> , <code>std::map</code>

Common Operations with Iterators

- 1.Dereferencing the Iterator (*it):** Accessing the element the iterator is pointing to.
- 2.Incrementing the Iterator (++it):** Moving to the next element.
- 3.Decrementing the Iterator (--it):** Moving to the previous element (available for bidirectional and random access iterators).
- 4.Comparison (it == end()):** Checking if the iterator has reached the end of the container.
- 5.Accessing an element directly (it[n]):** (Only for random access iterators like `std::vector`).

Example of using Iterator with vector

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<int> vec = {1, 2, 3, 4, 5};

    // Using iterator to traverse the vector
    cout << "Using iterators with vector:" << endl;
    for (vector<int>::iterator it = vec.begin(); it != vec.end(); ++it) {
        cout << *it << " ";
    }
    cout << endl;

    // Using range-based for loop (simplified)
    cout << "Using range-based for loop:" << endl;
    for (auto &val : vec) {
        cout << val << " ";
    }
    cout << endl;

    return 0;
}
```

Output:

```
Using iterators with vector:
1 2 3 4 5
Using range-based for loop:
1 2 3 4 5
```

Example of Iterators with list(Bidirectional iterator)

```
#include <iostream>
#include <list>
using namespace std;

int main() {
    list<int> myList = {10, 20, 30, 40};

    // Using iterator to traverse the list
    cout << "Using iterators with list:" << endl;
    for (list<int>::iterator it = myList.begin(); it != myList.end(); ++it) {
        cout << *it << " ";
    }
    cout << endl;

    // Using reverse iterator to traverse in reverse order
    cout << "Using reverse iterator with list:" << endl;
    for (list<int>::reverse_iterator rit = myList.rbegin(); rit != myList.rend(); ++rit) {
        cout << *rit << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

Output:

```
Using iterators with list:
10 20 30 40
Using reverse iterator with list:
40 30 20 10
```

Example of Iterators with std::map (Bidirectional & Random Access Iterator)

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<int, std::string> myMap = {{1, "One"}, {2, "Two"}, {3, "Three"}};

    // Using iterator to traverse the map
    cout << "Using iterators with map:" << endl;
    for (std::map<int, std::string>::iterator it = myMap.begin(); it != myMap.end(); ++it) {
        cout << it->first << ": " << it->second << endl;
    }

    return 0;
}
```

Output:

```
Using iterators with map:
1: One
2: Two
3: Three
```

Container Adaptors in C++

- C++ provides container adaptors, which are wrapper containers that modify the interface or behavior of other containers.
- They do not implement their own data structures but instead adapt other containers (like `std::queue`, `std::stack`, etc.) to meet specific needs.
- There are three main container adaptors in C++:
 - **`std::stack`** - Adapts containers to work as LIFO (Last In, First Out) stacks.
 - **`std::queue`** - Adapts containers to work as FIFO (First In, First Out) queues.
 - **`std::priority_queue`** - Adapts containers to work as priority queues (based on element priority).

`std::stack` (LIFO - Last In, First Out)

- `std::stack` is an adaptor container that allows access to elements in a LIFO (Last In, First Out) order.
- It is typically used for operations that follow the push-pop behavior, such as undo/redo operations, depth-first search, and more.
- Internal Container:
 - By default, `std::stack` uses `std::deque` as the underlying container. However, you can customize it to use other containers like `std::vector` or `std::list`.

Key Operations:

Method	Description
<code>push(value)</code>	Adds an element to the top of the stack.
<code>pop()</code>	Removes the top element from the stack.
<code>top()</code>	Returns the top element of the stack.
<code>empty()</code>	Returns <code>true</code> if the stack is empty, <code>false</code> otherwise.
<code>size()</code>	Returns the number of elements in the stack.

Example of Stack

```
#include <iostream>
#include <stack>

int main() {
    std::stack<int> s;

    s.push(10);
    s.push(20);
    s.push(30);

    std::cout << "Top: " << s.top() << std::endl; // Output: 30
    s.pop();
    std::cout << "Top after pop: " << s.top() << std::endl; //
Output: 20

    return 0;
}
```

Output:

```
Top: 30
Top after pop: 20
```

`std::queue` (FIFO - First In, First Out)

- `std::queue` is an adaptor container that implements a FIFO (First In, First Out) data structure, which is commonly used for tasks like task scheduling, buffering, and breadth-first search.
- Internal Container:
 - `std::queue` uses `std::deque` by default but can also work with other containers like `std::list`.

Key Operations:

Method	Description
<code>push(value)</code>	Adds an element to the end of the queue.
<code>pop()</code>	Removes the front element from the queue.
<code>front()</code>	Returns the front element of the queue.
<code>back()</code>	Returns the last element in the queue.
<code>empty()</code>	Returns <code>true</code> if the queue is empty, <code>false</code> otherwise.
<code>size()</code>	Returns the number of elements in the queue.

Example of Queue

```
#include <iostream>
#include <queue>

int main() {
    std::queue<int> q;

    q.push(10);
    q.push(20);
    q.push(30);

    std::cout << "Front: " << q.front() << std::endl; // Output:
10
    q.pop();
    std::cout << "Front after pop: " << q.front() << std::endl;
// Output: 20

    return 0;
}
```

Output:

```
Front: 10
Front after pop: 20
```

`std::priority_queue` (Priority Queue)

- `std::priority_queue` is an adaptor container that implements a priority queue, where elements are ordered based on their priority.
- By default, the highest priority element is the first to be removed. It's commonly used in algorithms like Dijkstra's algorithm and in task scheduling systems.
- Internal Container:
 - By default, `std::priority_queue` uses `std::vector` as the underlying container. However, it can be adapted to use other containers like `std::deque` or `std::list`.

Key Operations:

By default, `std::priority_queue` uses the greater-than operator (`>`) for comparisons, meaning larger elements have higher priority. You can customize the priority by passing a custom comparison function.

Method	Description
<code>push(value)</code>	Adds an element to the priority queue.
<code>pop()</code>	Removes the top element (highest priority).
<code>top()</code>	Returns the highest priority element.
<code>empty()</code>	Returns <code>true</code> if the priority queue is empty, <code>false</code> otherwise.
<code>size()</code>	Returns the number of elements in the priority queue.

Example of Priority queue

```
#include <iostream>
#include <queue>
#include <vector>

int main() {
    // Max-heap (default)
    std::priority_queue<int> pq;

    pq.push(10);
    pq.push(30);
    pq.push(20);

    std::cout << "Top: " << pq.top() << std::endl; // Output: 30
    pq.pop();
    std::cout << "Top after pop: " << pq.top() << std::endl; // Output: 20

    return 0;
}
```

Output:

```
Top: 30
Top after pop: 20
```

Lambda Expression in C++

- A **lambda expression** in C++ is an anonymous function that can be defined **inline** and used without the need to define a separate function.
- It allows you to write small, concise, and efficient functions for short-term use, making your code more readable and functional.

Syntax of Lambda Expression

Syntax:

```
[ capture_clause ] ( parameters ) -> return_type { function_body }
```

- **Capture Clause ([])**: Specifies which variables from the surrounding scope are available for use inside the lambda. It can capture by value or by reference.
- **Parameters ()**: List of parameters (similar to function arguments).
- **Return Type (-> return_type)**: Optional. Specifies the return type of the lambda.
- **Function Body ({ })**: The code that defines the functionality of the lambda expression.

Example

```
#include <iostream>

int main() {
    auto add = [](int a, int b) { return a + b; };
    std::cout << "Sum: " << add(3, 4) << std::endl; // Output: Sum: 7
    return 0;
}
```

Output:

Sum: 7

Example

```
#include <iostream>

int main() {
    int x = 10;
    auto printX = [x]() { std::cout << "Captured value: " << x << std::endl; };
    printX(); // Output: Captured value: 10
    return 0;
}
```

Output:

```
Captured value: 10
```

Example

```
#include <iostream>

int main() {
    int x = 10;
    auto modifyX = [&x]() { x = 20; };
    modifyX();
    std::cout << "Modified x: " << x << std::endl; // Output: Modified x: 20
    return 0;
}
```

Output:

Modified x: 20

Example

```
#include <iostream>

int main() {
    auto multiply = [](int a, int b) -> int { return a * b; };
    std::cout << "Product: " << multiply(3, 4) << std::endl; // Output: Product: 12
    return 0;
}
```

Output:

Product: 12